

**SRI CHANDRASEKHARENDRA SARASWATHI VISWA
MAHAVIDYALAYA**

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



Name : P. VEDA SAMHITHA

Reg. No : 11249A303

Class : II B.E. (CSE)

Course Code : BCSFC244A12

Course Name: Data Science for engineers lab

**SRI CHANDRASEKHARENDRA SARASWATHI VISWA
MAHAVIDYALAYA**

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by
Mr/Ms. P.VEDA SAMHITHA

with Reg. No 11249A303 of II-B.E.(CSE) during the academic year
2025 – 2026.

Station: Enathur

Date:

Staff-in-charge

Head of the Department

Submitted for the Practical examination held on _____.

Examiner-1

Examiner-2

SL.NO	ProgramName	Date	Page no	Signature
1	Introduction to python libraries			
a	Numpy			
b	Pandas			
c	Matplotlib			
d	Scikit-learn			
2	Perform data exploration and preprocessing python			
3	Implement Regularized Linear Regression			
4	Implement Naive Bayes Classifier			
5	Implement Regularized Logistic Regression			
6	Build models using ensembling techniques			
7	Build models using decision tree			
8	Support vector machines with different Kernels			
9	Implement K-Nearest Neighbours Classification			
10	K-Means clustering with PCA and elbow method			

Exp 1: Introduction to python libraries

1.1 NumPy

Aim: To implement array creation and operations using NumPy.

```
1 import numpy as np
   print(
       "1D Array:\n", np.array([1,2,3,4,5]), "\n\n"
       "2D Array:\n", np.array([[1,2,3],[4,5,6]]), "\n\n"
       "Zeros:\n", np.zeros((3,3)), "\n\n"
       "Ones:\n", np.ones((2,4)), "\n\n"
       "Identity:\n", np.eye(3), "\n\n"
       "Range:\n", np.arange(0,10,2)
   )

   import numpy as np
   a = np.array([1, 2, 3])
   b = np.array([4, 5, 6])
   print(
       "Array a:", a, "\n"
       "Array b:", b, "\n\n"
       "Addition (a + b):", a + b, "\n"
       "Multiplication (a * b):", a * b, "\n"
       "Scalar Multiplication (a * 10):", a * 10, "\n\n"
       "Square Root of a:", np.sqrt(a), "\n"
       "Mean of a:", np.mean(a), "\n"
       "Dot Product (a · b):", np.dot(a, b)
   )

   import numpy as np
   matrix = np.array([[10, 20, 30],
                      [40, 50, 60],
                      [70, 80, 90]])
   print(
       "Matrix:\n", matrix, "\n\n"
       "Element at [1,2]:", matrix[1, 2], "\n\n"
       "First Row:", matrix[0, :], "\n"
       "Second Column:", matrix[:, 1], "\n\n"
       "Top-left 2x2 Sub-matrix:\n", matrix[0:2, 0:2]
   )
```

```
1D Array:
[1 2 3 4 5]

...
2D Array:
[[1 2 3]
 [4 5 6]]

Zeros:
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]

Ones:
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]

Identity:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

Range:
[0 2 4 6 8]
Array a: [1 2 3]
Array b: [4 5 6]

Addition (a + b): [5 7 9]
Multiplication (a * b): [ 4 10 18]
Scalar Multiplication (a * 10): [10 20 30]

Square Root of a: [1.          1.41421356 1.73205081]
Mean of a: 2.0
Dot Product (a · b): 32

Matrix:
[[10 20 30]
 [40 50 60]
 [70 80 90]]

Element at [1,2]: 60

First Row: [10 20 30]
Second Column: [20 50 80]

Top-left 2x2 Sub-matrix:
[[10 20]
 [40 50]]
```

1.2 Pandas

Aim: To implement data creations, filtering, and cleaning using Pandas

```
[2] import pandas as pd
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Paris', 'London']
}
df = pd.DataFrame(data)
print(df.head())
print(df.info())
print(df.describe()), "\n\n"
ages = df['Age']
print(ages)
above_25 = df[df['Age'] > 25]
print(above_25)
row_0 = df.iloc[0]
print(row_0)
specific_val = df.loc[0, 'City']
print(specific_val)
print(df.isnull().sum()), "\n\n"
df_clean = df.dropna()
print(df_clean)
df_filled = df.fillna(0)
print(df_filled)
```

```

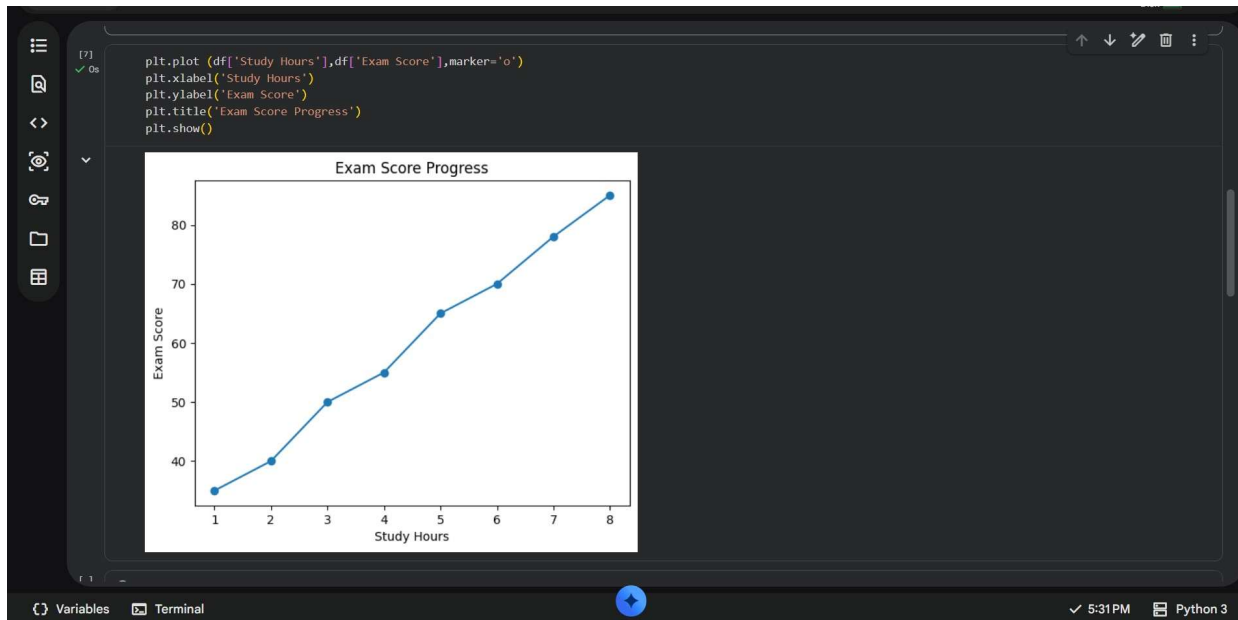
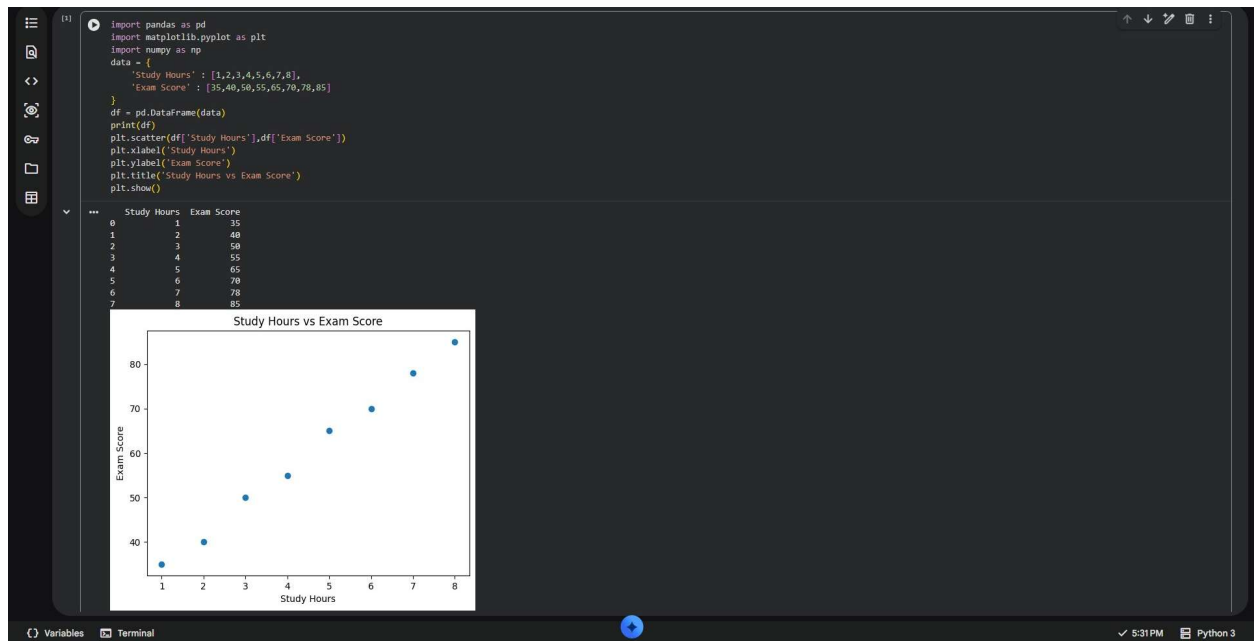
   Name  Age  City
0  Alice   25 New York
1   Bob   30   Paris
2 Charlie   35  London
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  -
0  Name      3 non-null      object
1  Age       3 non-null      int64
2  City      3 non-null      object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
```

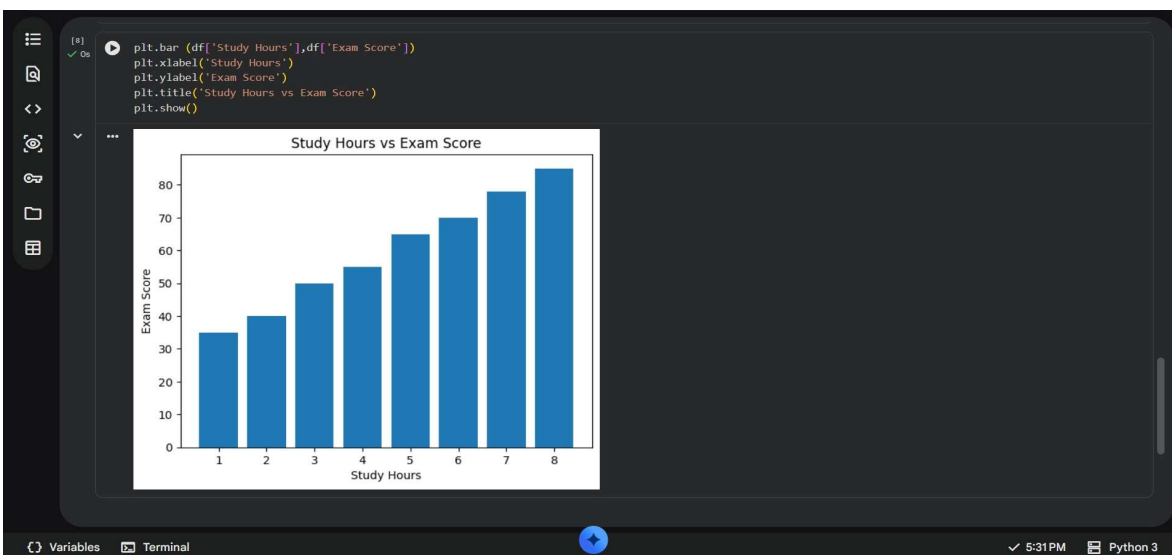
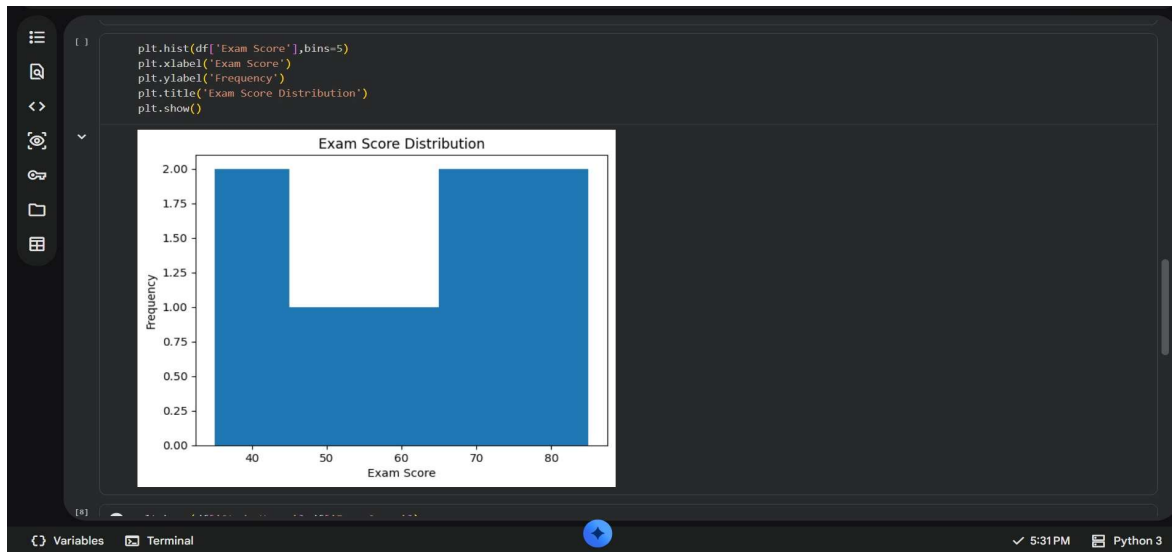
```

#   Column  Non-Null Count  Dtype
---  -
0  Name      3 non-null      object
1  Age       3 non-null      int64
2  City      3 non-null      object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
Age
count    3.0
mean    30.0
std      5.0
min     25.0
25%     27.5
50%     30.0
75%     32.5
max     35.0
0      25
1      30
2      35
Name: Age, dtype: int64
Name  Age  City
1   Bob   30   Paris
2 Charlie   35  London
Name  Age
Age    25
City   New York
Name: 0, dtype: object
New York
Name    0
Age     0
City    0
dtype: int64
   Name  Age  City
0  Alice   25 New York
1   Bob   30   Paris
2 Charlie   35  London
Name  Age  City
0  Alice   25 New York
1   Bob   30   Paris
2 Charlie   35  London
```

1.3 Matplot lib

Aim: To implement plotting operations using Matplotlib.





1.4 Scikit-Learn

Aim: To implement machine learning algorithms using Scikit-learn.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

data = {
    'Student' : ['S1','S2','S3','S4','S5','S6','S7','S8','S9','S10'],
    'classes_attended' : [30,35,40,45,50,55,60,65,70,75],
    'Internal_marks' : [35,38,42,45,47,50,52,55,58,60],
}

df = pd.DataFrame(data)
print(df)

X=df[['classes_attended']]
print(X)
y=df['Internal_marks']
print(y)

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=7)
print(X_train)
print(X_test)
print(y_train)
print(y_test)

model = LinearRegression()
model.fit(X_train,y_train)

predictions = model.predict(X_test)

mse = mean_squared_error(y_test, predictions)
print("MSE:", mse)
print("Marks per Class:", model.coef_[0])
print("Base Marks:", model.intercept_)

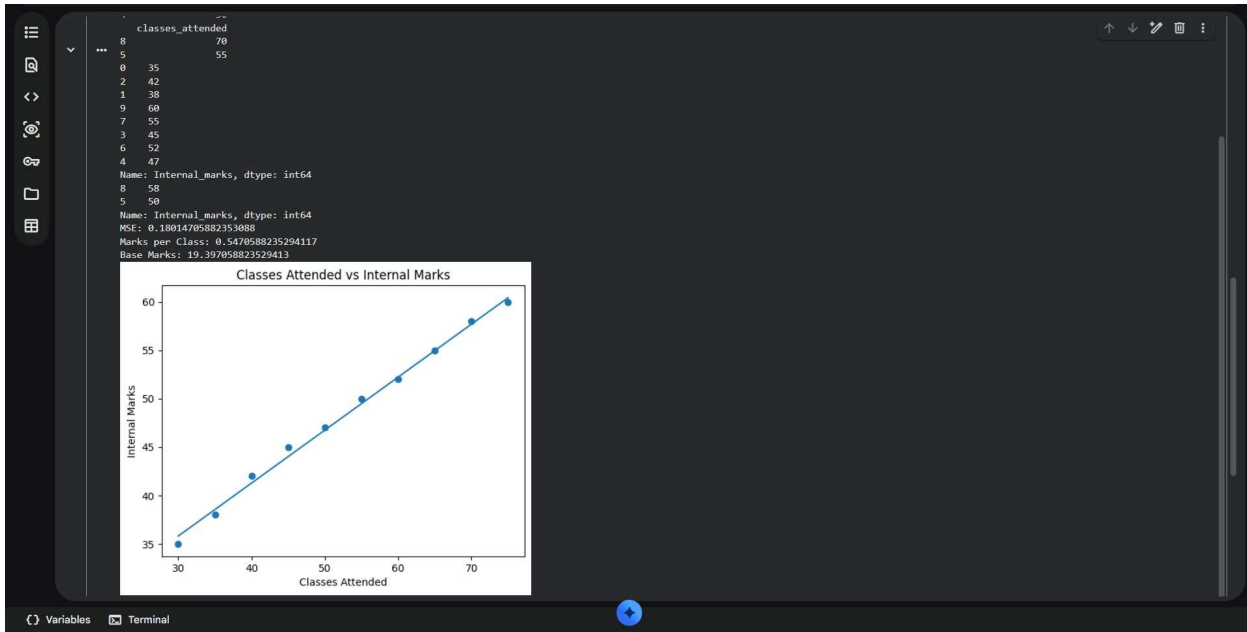
plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.xlabel("Classes Attended")
plt.ylabel("Internal Marks")
plt.title("Classes Attended vs Internal Marks")
plt.show()
```

```
Student  classes_attended  Internal_marks
0      S1                 30              35
1      S2                 35              38
2      S3                 40              42
3      S4                 45              45
4      S5                 50              47
5      S6                 55              50
6      S7                 60              52
7      S8                 65              55
8      S9                 70              58
9     S10                 75              60

classes_attended
0      30
1      35
2      40
3      45
4      50
5      55
6      60
7      65
8      70
9      75

0      35
1      38
2      42
3      45
4      47
5      50
6      52
7      55
8      58
9      60

Name: Internal_marks, dtype: int64
classes_attended
0      30
1      35
2      40
3      45
4      50
5      55
6      60
7      65
8      70
9      75
```

Result: Machine learning models were implemented successfully.

Exp 2 : Perform Data Exploration and preprocessing in python

Aim: To implement data exploration and preprocessing.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

data = {
    'Country': ['India', 'USA', 'India', 'USA', 'UK', 'India'],
    'Age': [22, 25, np.nan, 30, 28, 35],
    'Salary': [40000, np.nan, 50000, np.nan, 72000, 58000],
    'Purchased': ['No', 'Yes', 'Yes', 'No', 'Yes', 'Yes']
}

df = pd.DataFrame(data)
print("--- ORIGINAL RAW DATA ---")
print(df)
print("\n")

print("--- MISSING VALUES COUNT ---")
print(df.isnull().sum())
print("\n")
df['Age'] = df['Age'].fillna(df['Age'].mode())
df['Salary'] = df['Salary'].fillna(df['Salary'].mode())

print("--- DATA AFTER CLEANING ---")
print(df)
print("\n")
df_encoded = pd.get_dummies(df, columns=['Country'])
df_encoded['Purchased'] = df_encoded['Purchased'].map({'Yes': 1, 'No': 0})

print("--- DATA AFTER ENCODING (All Numbers Now) ---")
print(df_encoded)
print("\n")

X = df_encoded.drop('Purchased', axis=1)
y = df_encoded['Purchased']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("--- FINAL PROCESSED DATA (Ready for AI) ---")
print(pd.DataFrame(X_scaled, columns=X.columns).head())
```

```
--- ORIGINAL RAW DATA ---
Country  Age  Salary  Purchased
0  India  22.0  40000.0         No
1  USA    25.0      NaN         Yes
2  India  NaN    50000.0         Yes
3  USA    30.0      NaN         No
4  UK     28.0  72000.0         Yes
5  India  35.0  58000.0         Yes

--- MISSING VALUES COUNT ---
Country    0
Age         1
Salary      2
Purchased   0
dtype: int64

--- DATA AFTER CLEANING ---
Country  Age  Salary  Purchased
0  India  22.0  40000.0         No
1  USA    25.0  50000.0         Yes
2  India  28.0  50000.0         Yes
3  USA    30.0  72000.0         No
4  UK     28.0  72000.0         Yes
5  India  35.0  58000.0         Yes

--- DATA AFTER ENCODING (All Numbers Now) ---
   Age  Salary  Purchased  Country_India  Country_UK  Country_USA
0  22.0  40000.0          0             True         False         False
1  25.0  50000.0          1             False         False          True
2  28.0  50000.0          1             True         False         False
3  30.0  72000.0          0             False         False          True
4  28.0  72000.0          1             False          True         False
5  35.0  58000.0          1             True          False         False

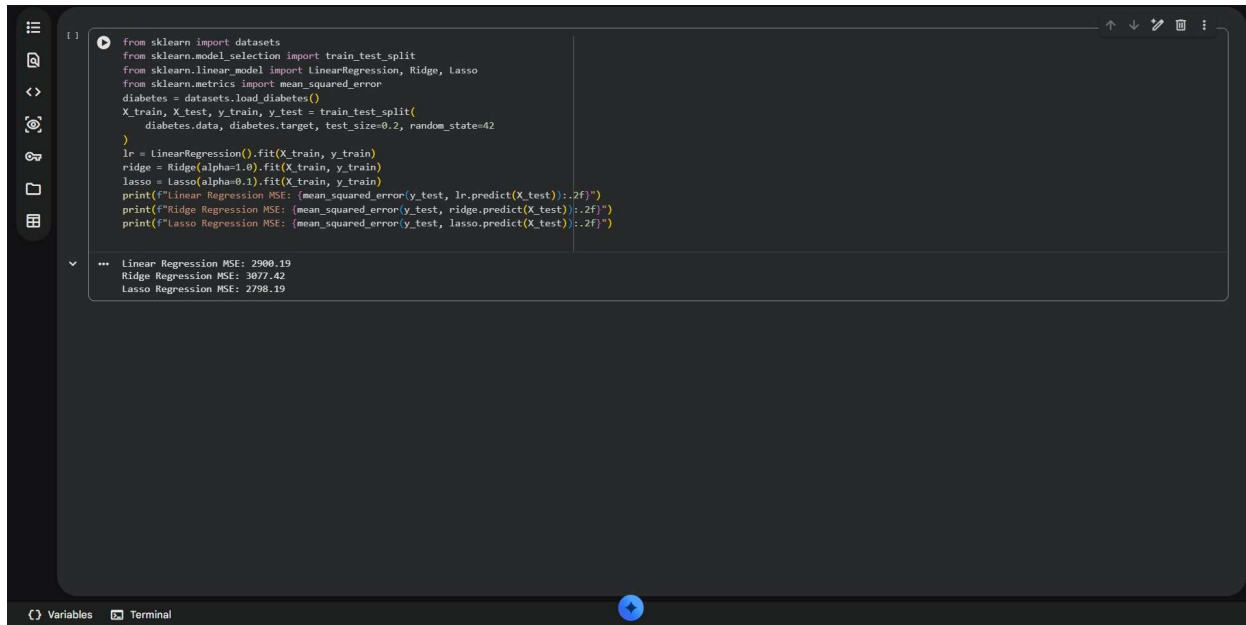
--- FINAL PROCESSED DATA (Ready for AI) ---
   Age  Salary  Country_India  Country_UK  Country_USA
0 -1.486615 -1.430476          1.0    -0.447214   -0.707107
1 -0.742307 -0.592314         -1.0    -0.447214   1.414214
2  0.000000 -0.592314          1.0    -0.447214   -0.707107
3  0.494872  1.269243         -1.0    -0.447214   1.414214
4  0.000000  1.269243         -1.0    2.236068   -0.707107
```

Result : Data preprocessing was performed successfully.

Exp

3 : Implement Regularized Linear Regression

Aim: To implement regularized linear regression.



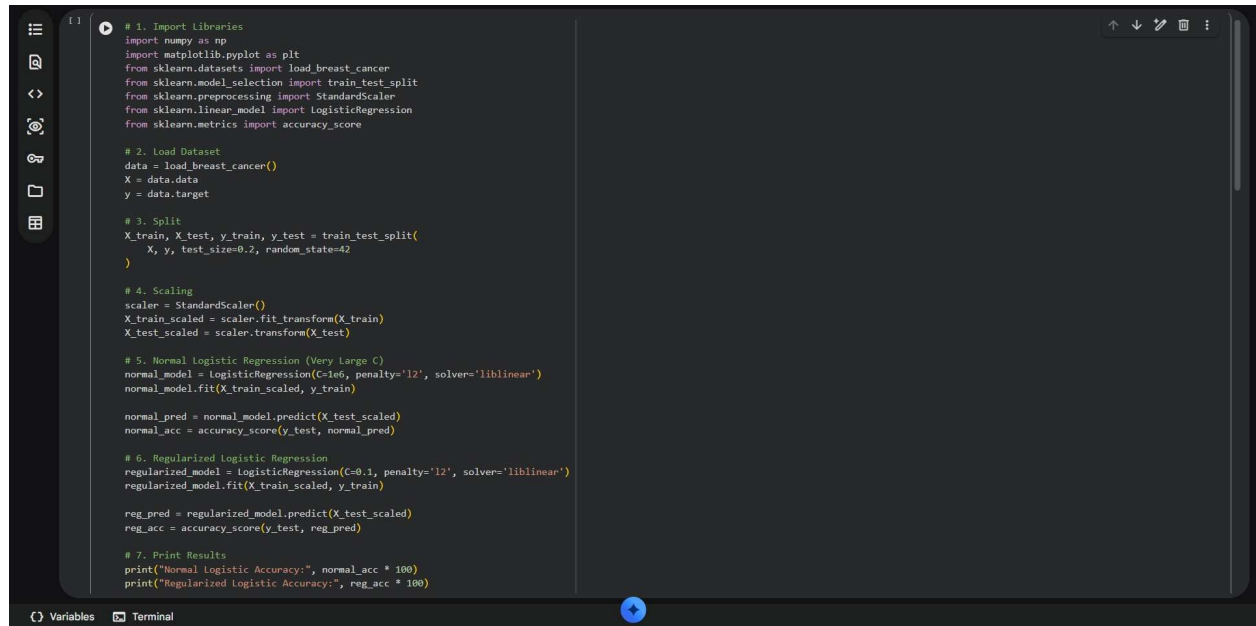
```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error
diabetes = datasets.load_diabetes()
X_train, X_test, y_train, y_test = train_test_split(
    diabetes.data, diabetes.target, test_size=0.2, random_state=42
)
lr = LinearRegression().fit(X_train, y_train)
ridge = Ridge(alpha=1.0).fit(X_train, y_train)
lasso = Lasso(alpha=0.1).fit(X_train, y_train)
print("Linear Regression MSE: (mean_squared_error(y_test, lr.predict(X_test)):.2f)")
print("Ridge Regression MSE: (mean_squared_error(y_test, ridge.predict(X_test)):.2f)")
print("Lasso Regression MSE: (mean_squared_error(y_test, lasso.predict(X_test)):.2f)")
```

Linear Regression MSE: 2900.19
Ridge Regression MSE: 3077.42
Lasso Regression MSE: 2798.19

Result : The regression model was implemented successfully

Exp 4 : Implement Naive Bayes Classifier

Aim: To implement Navie Bayes Classification.



```
# 1. Import Libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 2. Load Dataset
data = load_breast_cancer()
X = data.data
y = data.target

# 3. Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4. Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 5. Normal Logistic Regression (Very Large C)
normal_model = LogisticRegression(C=1e6, penalty='l2', solver='liblinear')
normal_model.fit(X_train_scaled, y_train)

normal_pred = normal_model.predict(X_test_scaled)
normal_acc = accuracy_score(y_test, normal_pred)

# 6. Regularized Logistic Regression
regularized_model = LogisticRegression(C=0.1, penalty='l2', solver='liblinear')
regularized_model.fit(X_train_scaled, y_train)

reg_pred = regularized_model.predict(X_test_scaled)
reg_acc = accuracy_score(y_test, reg_pred)

# 7. Print Results
print("Normal Logistic Accuracy:", normal_acc * 100)
print("Regularized Logistic Accuracy:", reg_acc * 100)
```

Result: The Classifier was implemented successfully.

5:-Implement Regularised logistic regression

Aim: To implement regularized logistic regression.

Exp

```
# 1. Import Libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 2. Load Dataset
data = load_breast_cancer()
X = data.data
y = data.target

# 3. Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4. Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

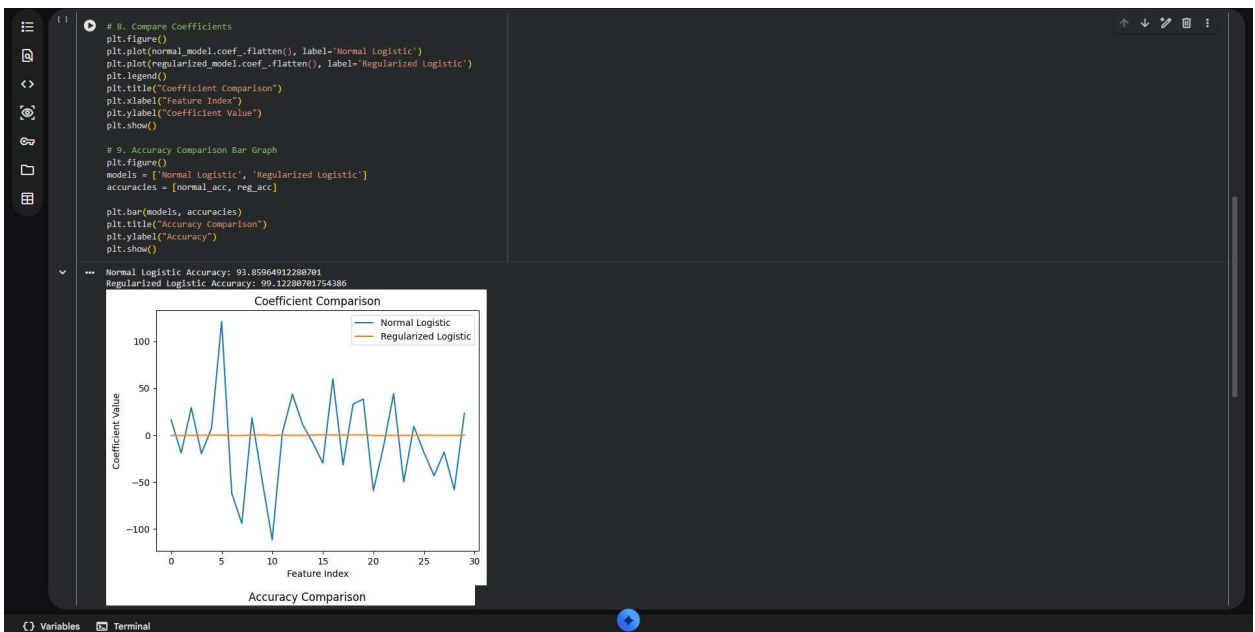
# 5. Normal Logistic Regression (Very Large C)
normal_model = LogisticRegression(C=1e6, penalty='l2', solver='liblinear')
normal_model.fit(X_train_scaled, y_train)

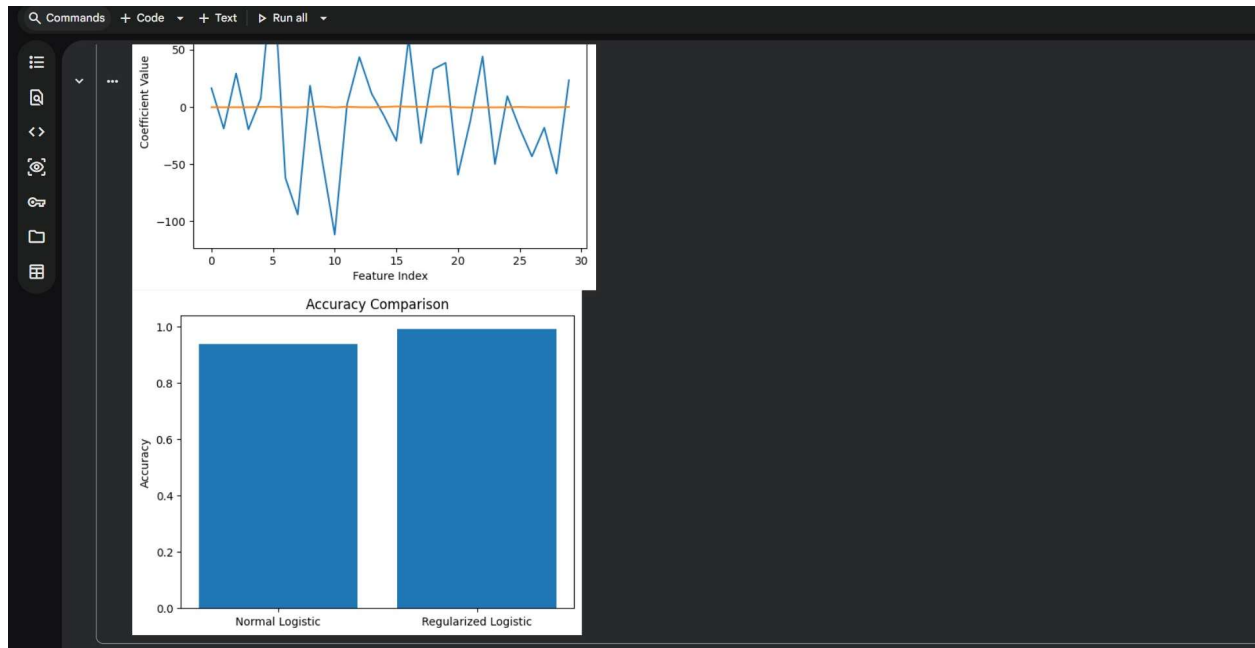
normal_pred = normal_model.predict(X_test_scaled)
normal_acc = accuracy_score(y_test, normal_pred)

# 6. Regularized Logistic Regression
regularized_model = LogisticRegression(C=0.1, penalty='l2', solver='liblinear')
regularized_model.fit(X_train_scaled, y_train)

reg_pred = regularized_model.predict(X_test_scaled)
reg_acc = accuracy_score(y_test, reg_pred)

# 7. Print Results
print("Normal Logistic Accuracy:", normal_acc * 100)
print("Regularized Logistic Accuracy:", reg_acc * 100)
```



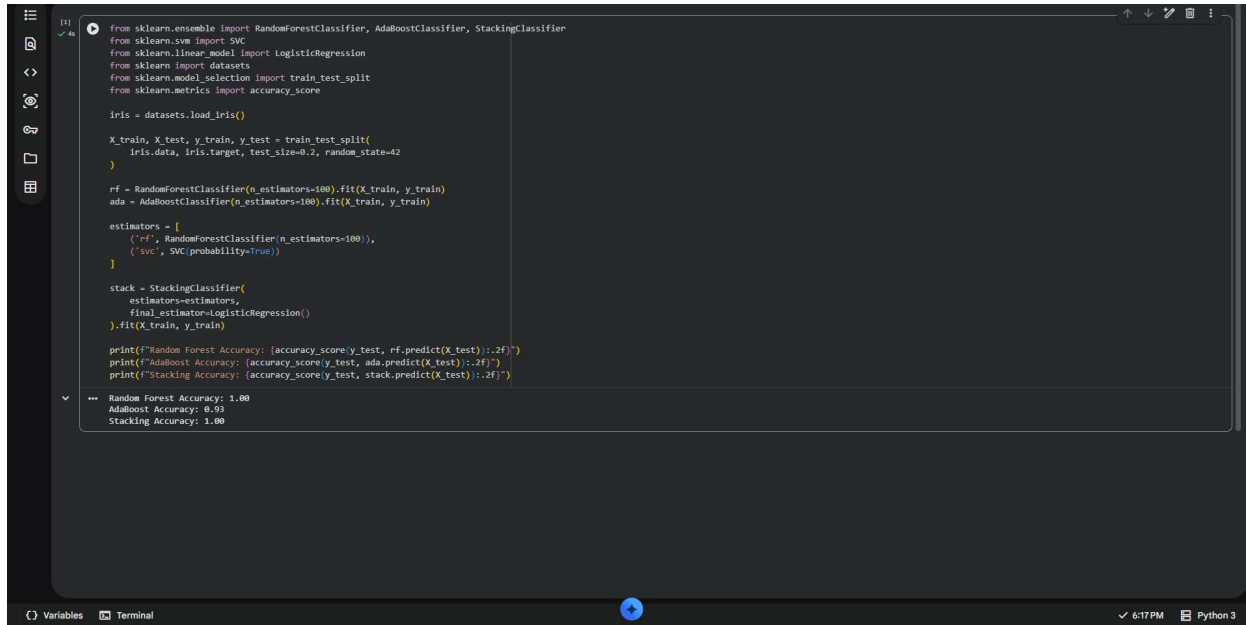


Result: The logistic regression model was implemented successfully.

6 : Build Models using different Ensembling Techniques

Aim: To implement ensembling techniques.

Exp



```
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

Iris = datasets.load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    Iris.data, Iris.target, test_size=0.2, random_state=42
)

rf = RandomForestClassifier(n_estimators=100).fit(X_train, y_train)
ada = AdaBoostClassifier(n_estimators=100).fit(X_train, y_train)

estimators = [
    ('rf', RandomForestClassifier(n_estimators=100)),
    ('svc', SVC(probability=True))
]

stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
).fit(X_train, y_train)

print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
```

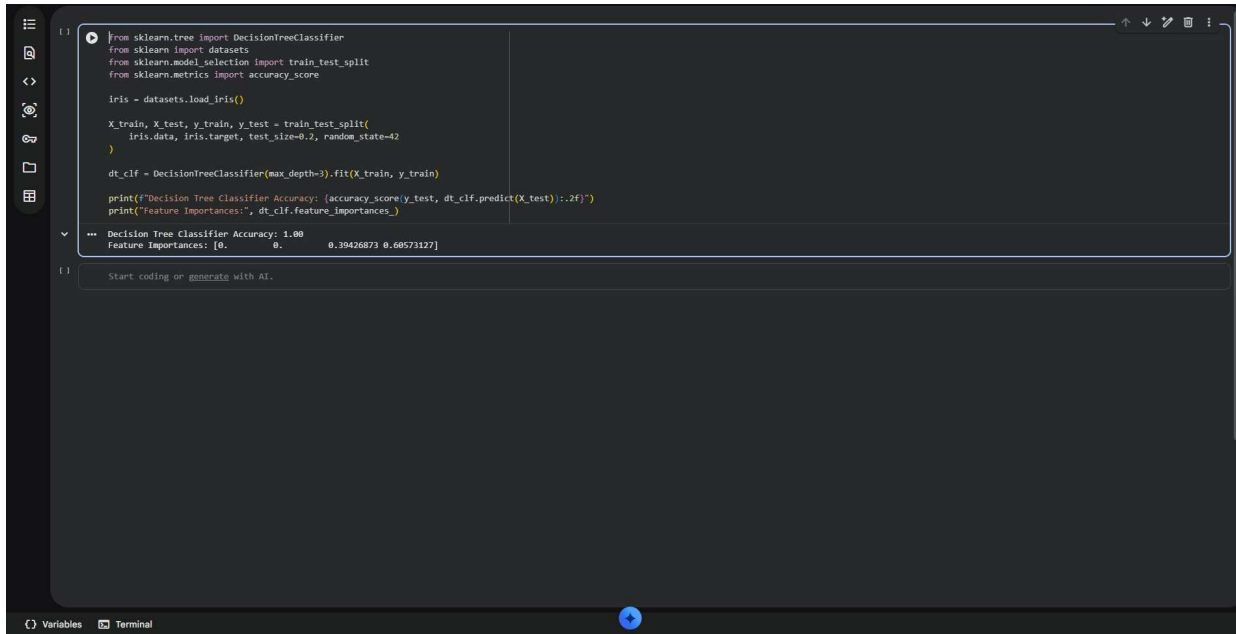
Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00

Result: The ensembling techniques were implemented successfully.

Exp

7 : Build Models using Decision Trees

Aim: To implement decision tree models.



```
from sklearn.tree import DecisionTreeClassifier
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

iris = datasets.load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)

dt_clf = DecisionTreeClassifier(max_depth=3).fit(X_train, y_train)

print(f'Decision Tree Classifier Accuracy: {accuracy_score(y_test, dt_clf.predict(X_test)):.2f}')
print('Feature Importances:', dt_clf.feature_importances_)

--- Decision Tree Classifier Accuracy: 1.00
Feature Importances: [0. 0. 0.39426873 0.60573127]
```

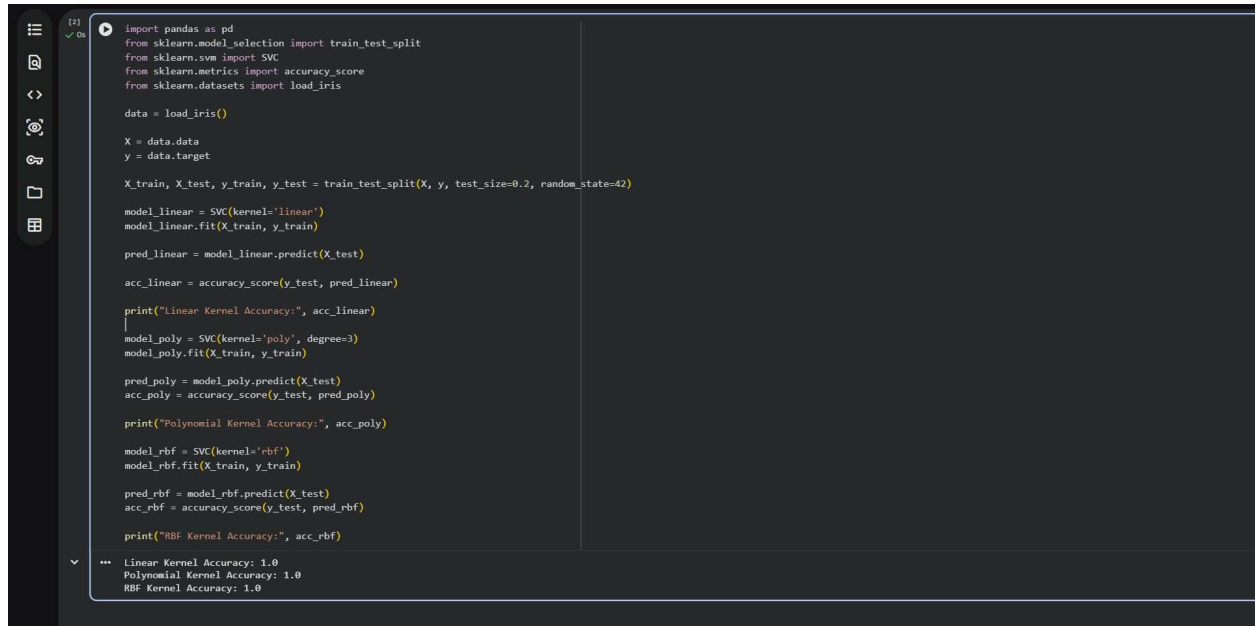
Start coding or generate with AI.

Result: The decision tree models were implemented successfully.

Exp

8: Support Vector Machines (SVM) with various Kernels

Aim: To implement Support Vector Machine.



```
[In] 0a
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris

data = load_iris()

X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model_linear = SVC(kernel='linear')
model_linear.fit(X_train, y_train)

pred_linear = model_linear.predict(X_test)

acc_linear = accuracy_score(y_test, pred_linear)

print("Linear Kernel Accuracy:", acc_linear)
|
model_poly = SVC(kernel='poly', degree=3)
model_poly.fit(X_train, y_train)

pred_poly = model_poly.predict(X_test)
acc_poly = accuracy_score(y_test, pred_poly)

print("Polynomial Kernel Accuracy:", acc_poly)

model_rbf = SVC(kernel='rbf')
model_rbf.fit(X_train, y_train)

pred_rbf = model_rbf.predict(X_test)
acc_rbf = accuracy_score(y_test, pred_rbf)

print("RBF Kernel Accuracy:", acc_rbf)
```

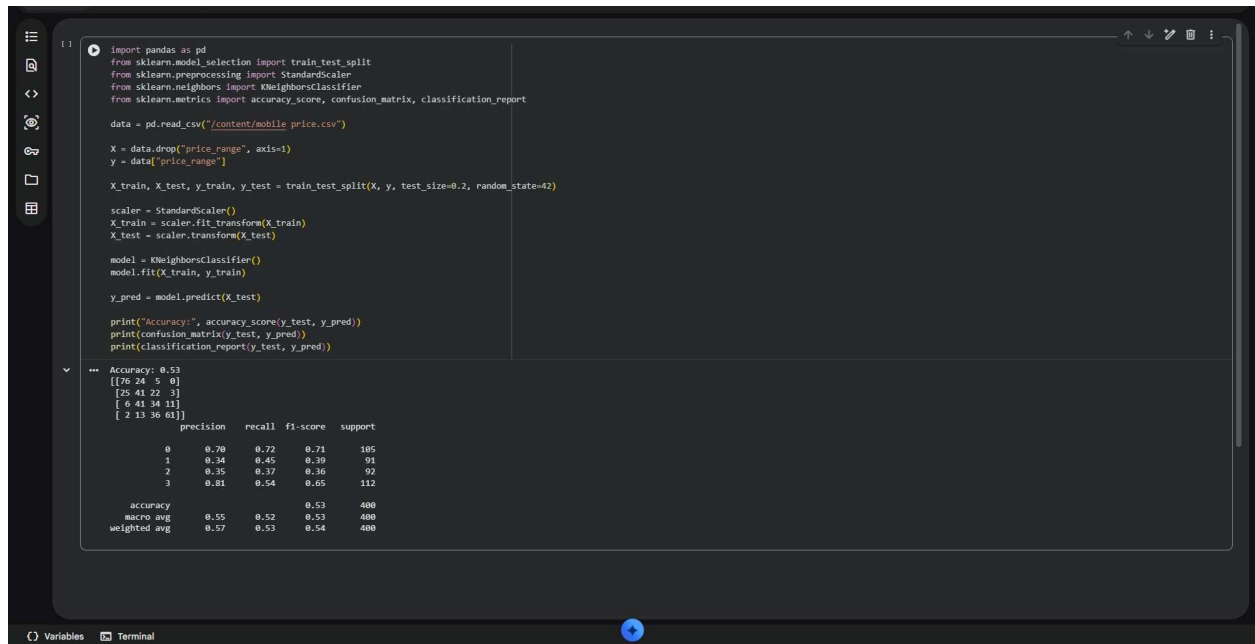
... Linear Kernel Accuracy: 1.0
Polynomial Kernel Accuracy: 1.0
RBF Kernel Accuracy: 1.0

Result: The Support Vector Machines with Kernels was implemented successfully

9:-Implement KNN Classification

Exp

Aim: To implement K-Nearest Neighbors algorithm



The screenshot shows a Jupyter Notebook interface with a code cell and its output. The code implements a K-Nearest Neighbors (KNN) classification model using the sklearn library. It reads data from a CSV file, preprocesses it with StandardScaler, and evaluates the model's performance using accuracy, confusion matrix, and classification report.

```
11 import pandas as pd
12 from sklearn.model_selection import train_test_split
13 from sklearn.preprocessing import StandardScaler
14 from sklearn.neighbors import KNeighborsClassifier
15 from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
16
17 data = pd.read_csv("/content/mobile_price.csv")
18 X = data.drop("price_range", axis=1)
19 y = data["price_range"]
20
21 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
22
23 scaler = StandardScaler()
24 X_train = scaler.fit_transform(X_train)
25 X_test = scaler.transform(X_test)
26
27 model = KNeighborsClassifier()
28 model.fit(X_train, y_train)
29
30 y_pred = model.predict(X_test)
31
32 print("Accuracy:", accuracy_score(y_test, y_pred))
33 print(confusion_matrix(y_test, y_pred))
34 print(classification_report(y_test, y_pred))
```

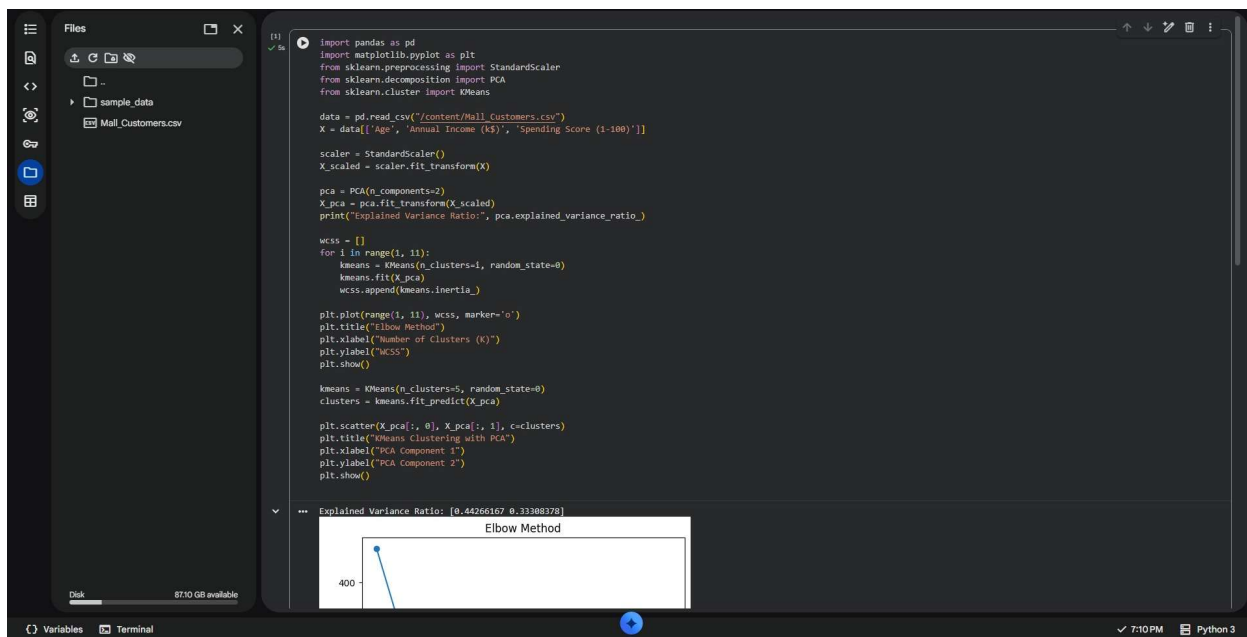
Accuracy: 0.53
[[76 24 5 0]
[25 41 22 3]
[6 41 34 11]
[2 13 36 61]]

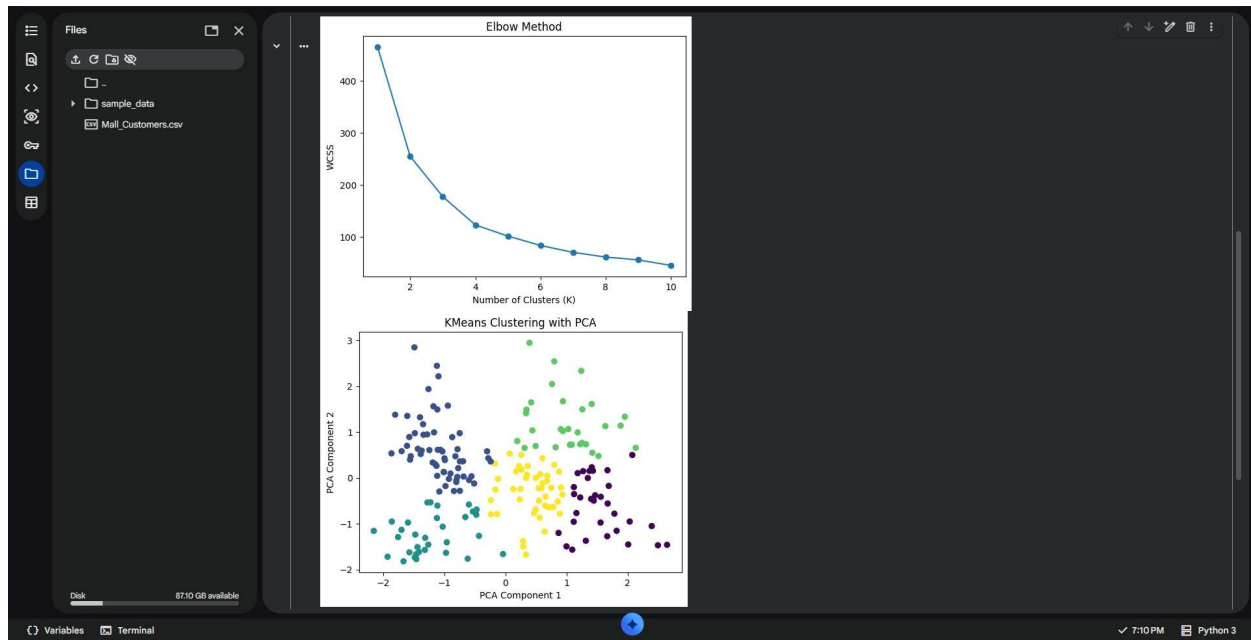
	precision	recall	f1-score	support
0	0.70	0.72	0.71	105
1	0.34	0.45	0.39	91
2	0.35	0.17	0.26	92
3	0.81	0.54	0.65	112
accuracy			0.53	400
macro avg	0.55	0.52	0.53	400
weighted avg	0.57	0.53	0.54	400

Result: The KNN Classification model was implemented successfully.

Exp 10 : K- Means Clustering with PCA and Elbow Method

Aim: To implement k-means clustering.





Result: K- Means Clustering with PCA and Elbow Method was performed successfully.